Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК5 «Системы обработки информации»***

Лабораторная работа № 5

«Uplift-моделирование»

ДИСЦИПЛИНА: «Технологии обработки больших данных»

Выполнил: студент гр.ИУК5-82Б _____ (Ли Р.В. _____)
(Подпись) (Ф.И.О.)

Проверил: _____ (_____)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

Калуга, 2026

Цель лабораторной работы — освоить постановку uplift-моделирования и базовый пайплайн построения uplift-моделей для оценки эффекта воздействия (*treatment*) на вероятность целевого действия (*target*) на примере датасета email-кампании (Hillstrom).

Задачи:

- понять, чем uplift-моделирование отличается от обычной классификации; разобрать переменные X, t, y и формулу uplift как разницу матожиданий при *t=1* и *t=0*;
- загрузить данные (Hillstrom), выделить признаки X**,** воздействие t**,** целевую переменную y, выполнить первичный просмотр;
- выполнить подготовку данных: корректно сделать train/test split (без утечки) и закодировать категориальные признаки (One-Hot Encoding);
- обучить и сравнить несколько подходов uplift-моделирования (например, S-learner / SoloModel**,** ClassTransformation**,** T-learner / TwoModels) на одной базовой модели;
- оценить качество uplift-моделей с помощью метрик и визуализаций из scikit-uplift (например, uplift_at_k, uplift-кривые и графики предсказаний), сделать вывод о лучшем подходе.

Ход выполнения:

[2]:
```
from sklift.datasets import fetch_hillstrom

data = fetch_hillstrom(target_col='visit')
data.keys()
```

[2]:
```
dict_keys(['data', 'target', 'treatment', 'DESCR', 'feature_names', 'target_name', 'treatment_name'])
```

[3]:
```
X = data['data']
t = data['treatment']
y = data['target']
X.head()
t.value_counts()
y.value_counts()
```

[3]:
```
visit
0    54606
1     9394
Name: count, dtype: int64
```

[4]:
```
print("Treatment distribution:")
print(t.value_counts(normalize=True))

print("\nTarget distribution:")
print(y.value_counts(normalize=True))
```

```
Treatment distribution:
segment
Womens E-Mail    0.334172
Mens E-Mail      0.332922
No E-Mail        0.332906
Name: proportion, dtype: float64

Target distribution:
visit
0    0.853219
1    0.146781
Name: proportion, dtype: float64
```

[6]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, t_train, t_test, y_train, y_test = train_test_split(
    X, t, y,
    test_size=0.3,
    random_state=42,
    stratify=t
)
```

[7]:
```
cat_cols = X.select_dtypes(include=['object']).columns
cat_cols
```

```
/tmp/ipykernel_14924/241291938.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dty
pe is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or t
o `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that w
orks with pandas 2 and 3.
  cat_cols = X.select_dtypes(include=['object']).columns
```

[7]:
```
Index(['history_segment', 'zip_code', 'channel'], dtype='str')
```

[11]:
```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

encoder.fit(X_train[cat_cols])

X_train_cat = encoder.transform(X_train[cat_cols])
X_test_cat = encoder.transform(X_test[cat_cols])
num_cols = X.select_dtypes(exclude=['object']).columns

X_train_num = X_train[num_cols].values
X_test_num = X_test[num_cols].values
import numpy as np

X_train_final = np.hstack([X_train_num, X_train_cat])
X_test_final = np.hstack([X_test_num, X_test_cat])
```

[17]:
```
X_train_final.shape
X_test_final.shape
t = t.apply(lambda x: 0 if x == 'No E-Mail' else 1)
```

[18]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, t_train, t_test, y_train, y_test = train_test_split(
    X, t, y,
    test_size=0.3,
    random_state=42,
    stratify=t
)
```

[19]:
```
# encoding
encoder.fit(X_train[cat_cols])
X_train_cat = encoder.transform(X_train[cat_cols])
X_test_cat = encoder.transform(X_test[cat_cols])

X_train_num = X_train[num_cols].values
X_test_num = X_test[num_cols].values

X_train_final = np.hstack([X_train_num, X_train_cat])
X_test_final = np.hstack([X_test_num, X_test_cat])
```

[20]:
```
s_learner.fit(X_train_final, y_train, t_train)
```

[20]:

▸ **SoloModel** ⓘ

▸ **estimator: RandomForestClassifier**

▸ RandomForestClassifier ⓘ

[21]:
```
uplift_s = s_learner.predict(X_test_final)
print(type(uplift_s))
print(uplift_s.shape)
print(uplift_s[:5])
```

```
<class 'numpy.ndarray'>
(19200,)
[ 0.11  0.    0.68  0.11 -0.03]
```

[22]:
```
print(len(y_test), len(t_test), len(uplift_s))
```

```
19200 19200 19200
```

[23]:
```
from sklift.metrics import uplift_at_k

score_s = uplift_at_k(y_test, uplift_s, t_test, strategy='by_group', k=0.3)
print("S-learner uplift@30%:", score_s)
```

```
S-learner uplift@30%: 0.0421499246311734
```

[24]:
```
from sklift.models import TwoModels
from sklearn.ensemble import RandomForestClassifier

t_learner = TwoModels(
    estimator_trmnt=RandomForestClassifier(n_estimators=100, random_state=42),
    estimator_ctrl=RandomForestClassifier(n_estimators=100, random_state=42)
)

t_learner.fit(X_train_final, y_train, t_train)
```

[24]:

▸ **TwoModels** ⓘ

▸ **estimator_ctrl: RandomForestClassifier** ▸ **estimator_trmnt: RandomForestClassifier**

▸ RandomForestClassifier ⓘ ▸ RandomForestClassifier ⓘ

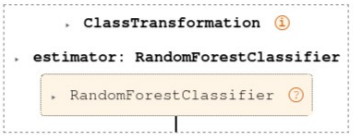
[26]:
```
uplift_t = t_learner.predict(X_test_final)
print(uplift_t.shape)
print(uplift_t[:5])
```

```
(19200,)
[ 0.17  0.    0.71  0.2  -0.01]
```

[27]:
```
from sklift.models import ClassTransformation

ct = ClassTransformation(
    estimator=RandomForestClassifier(n_estimators=100, random_state=42)
)

ct.fit(X_train_final, y_train, t_train)
```



[28]:
```
uplift_ct = ct.predict(X_test_final)
print(uplift_ct.shape)
print(uplift_ct[:5])
```

```
(19200,)
[ 0.6  -0.98  0.9  -0.72 -0.4 ]
```

[29]:
```
from sklift.metrics import uplift_at_k

print("S-learner:", uplift_at_k(y_test, uplift_s, t_test, strategy='by_group', k=0.3))
print("T-learner:", uplift_at_k(y_test, uplift_t, t_test, strategy='by_group', k=0.3))
print("ClassTransformation:", uplift_at_k(y_test, uplift_ct, t_test, strategy='by_group', k=0.3))
```

```
S-learner: 0.0421499246311734
T-learner: 0.04241020573476528
ClassTransformation: 0.04763212083343178
```

[31]:
```
from sklift.metrics import uplift_at_k

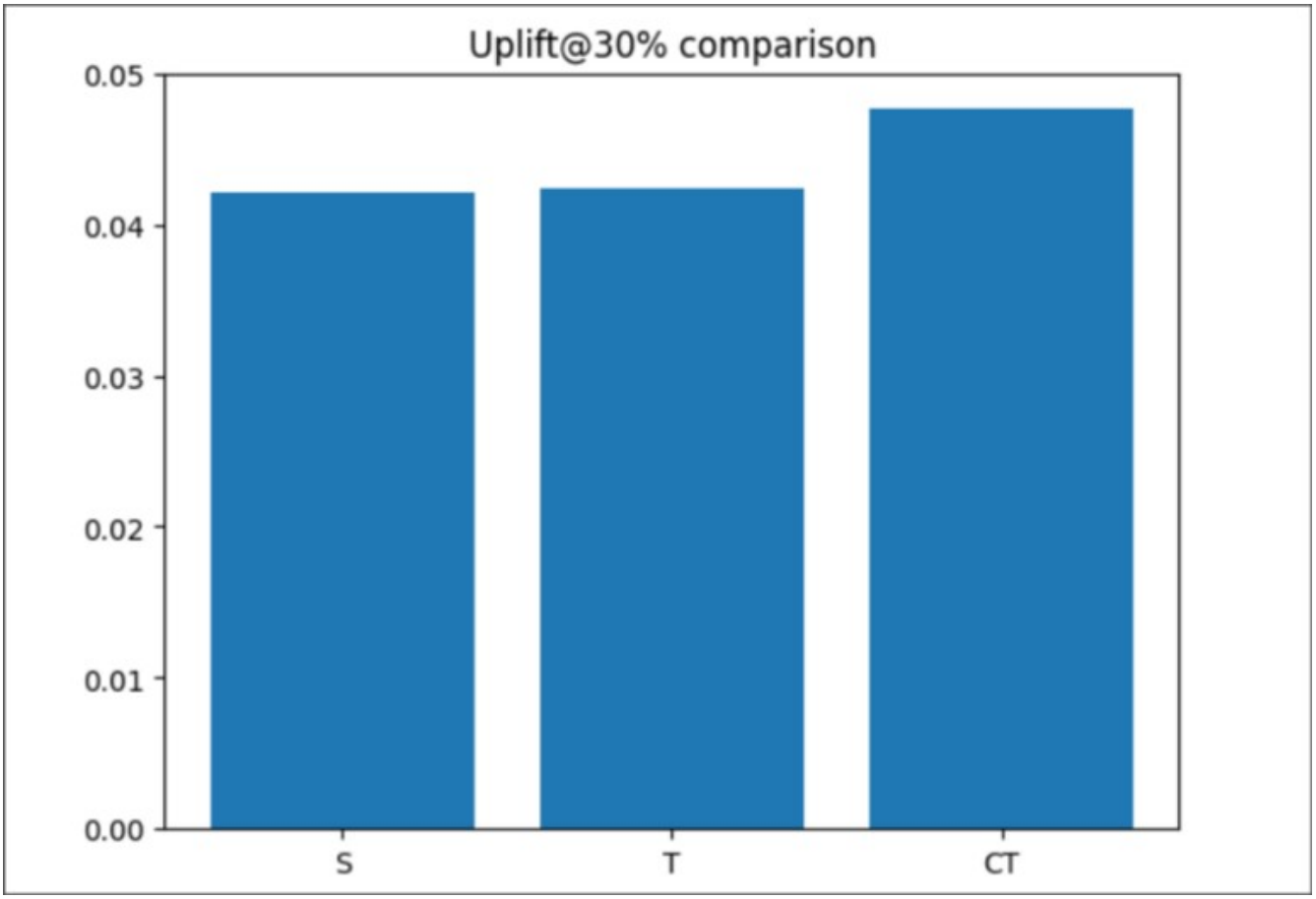
print("S-learner:", uplift_at_k(y_test, uplift_s, t_test, strategy='by_group', k=0.3))
print("T-learner:", uplift_at_k(y_test, uplift_t, t_test, strategy='by_group', k=0.3))
print("ClassTransformation:", uplift_at_k(y_test, uplift_ct, t_test, strategy='by_group', k=0.3))

import matplotlib.pyplot as plt

models = ["S", "T", "CT"]
scores = [
    uplift_at_k(y_test, uplift_s, t_test, strategy='by_group', k=0.3),
    uplift_at_k(y_test, uplift_t, t_test, strategy='by_group', k=0.3),
    uplift_at_k(y_test, uplift_ct, t_test, strategy='by_group', k=0.3),
]

plt.bar(models, scores)
plt.title("Uplift@30% comparison")
plt.show()
```

```
S-learner: 0.0421499246311734
T-learner: 0.04241020573476528
ClassTransformation: 0.04763212083343178
```



Вывод: были изучены ансамблевые методы бэггинга и градиентного бустинга, а также на практике сравнить реализации бустинга (XGBoost, LightGBM, CatBoost) с RandomForest на задачах классификации и регрессии, включая базовую предобработку и подбор гиперпараметров.